

ALGORITHMS & COMPLEXITY

LECTURE 4

Sometimes if the counting (of memory consumption, of number of basic operations etc) is given one does not need to re-transform recursive scheme from implicit form $\Rightarrow$ explicit form

Example 1:

$$* \begin{cases} a_n = a_{n-2} + a_{n-3} & n \geq 3 \\ a_0 = a_1 = a_2 = 1 \end{cases}$$

We search for some upper bound to "assess" the estimate of the rate of "explosion" of a_n . One can plot on a computer pairs:

$$(n_0, a_{n_0}), (n_0+1, a_{n_0+1}), \dots,$$

& try to compare with plots of curves $\beta \ln x$, βx^n , βx^α , $\beta \sqrt[n]{n}$, βa^n , etc.

this helps to formulate the hypothesis on upper bound which can be inductively proved & also later improved.

ALGORITHMS & COMPLEXITY

LECTURE 4

In case of (*) a_n take the values

1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 21, 28, 37, 49, ...

We set up the hypothesis (this is the hardest here as proving it is easier)

$$P(n): 0 \leq a_n \leq \left(\frac{4}{3}\right)^n$$

We prove now $P(n)$ by using M. Induction

I. We verify $P(0)$, $P(1)$, $P(2)$

$$P(0): 0 \leq a_0 = 1 \leq \left(\frac{4}{3}\right)^0 = 1 \quad \text{yes}$$

$$P(1): 0 \leq a_1 = 1 \leq \left(\frac{4}{3}\right)^1 = \frac{4}{3} \quad \text{yes}$$

$$P(2): 0 \leq a_2 = 1 \leq \left(\frac{4}{3}\right)^2 = \frac{16}{9} \quad \text{yes}$$

II Assume now

$$(A) \begin{cases} P(n-1): 0 \leq a_{n-1} \leq \left(\frac{4}{3}\right)^{n-1} \\ P(n-2): 0 \leq a_{n-2} \leq \left(\frac{4}{3}\right)^{n-2} \\ P(n-3): 0 \leq a_{n-3} \leq \left(\frac{4}{3}\right)^{n-3} \end{cases}$$

are all true.

ALGORITHMS & COMPLEXITY

LECTURE 4

we want to show that from (A)

we can infer:

$$P(n): \quad 0 \leq a_n \leq \left(\frac{4}{3}\right)^n$$

By recursive formula

$$a_n = a_{n-2} + a_{n-3}$$

since by (A) $a_{n-2} \geq 0$ & $a_{n-3} \geq 0$

$a_n \geq 0$. (by (A)). Furthermore:

$$\begin{aligned} a_n = a_{n-2} + a_{n-3} &\stackrel{\text{by (A)}}{\leq} \left(\frac{4}{3}\right)^{n-2} + \left(\frac{4}{3}\right)^{n-3} \\ &= \underbrace{\left(\frac{4}{3}\right)^2 \cdot \left(\frac{3}{4}\right)^2}_{\text{as } 1} \cdot \left(\frac{4}{3}\right)^{n-2} + \underbrace{\left(\frac{4}{3}\right)^3 \cdot \left(\frac{3}{4}\right)^3}_{\text{as } 1} \left(\frac{4}{3}\right)^{n-3} \end{aligned}$$

$$= \left(\frac{4}{3}\right)^n \cdot \frac{9}{16} + \left(\frac{4}{3}\right)^n \cdot \frac{27}{64} = \left(\frac{4}{3}\right)^n \left(\frac{36}{64} + \frac{27}{64}\right)$$

$$= \left(\frac{4}{3}\right)^n \cdot \frac{63}{64} < \left(\frac{4}{3}\right)^n \Rightarrow \underbrace{0 \leq a_n \leq \left(\frac{4}{3}\right)^n}_{P(n)}$$

ALGORITHMS & COMPLEXITY

LECTURE 4

Thus by P.M.I (principle of mathematical induction):

$$0 \leq a_n \leq \left(\frac{4}{3}\right)^n \quad \forall n \geq 0$$

$$\Downarrow \\ a_n = O\left(\left(\frac{4}{3}\right)^n\right)$$

Is it sharp?

If not can we improve it i.e. find $f(n)$ s. that $a_n = O(f(n))$ where

$0 \leq f(n) < \left(\frac{4}{3}\right)^n$. What is the sharp result i.e. $a_n = \Theta(f(n))$.

All such questions are standard in the analysis of the algorithm's complexity. The answers are usually not trivial & each time require a separate analysis. To some class of recursive scheme (common in computer science)

one can derive a general theory:
implicit form $\xrightarrow{\text{how to transform}}$ explicit form

ALGORITHMS & COMPLEXITY

LECTURE 4

Sometimes: one chooses from

$a_0, a_1, a_2, a_3, \dots, a_n, a_{n+1}, \dots$

only some terms as it may be easier to analyze them

$a_{k_1}, a_{k_2}, \dots, a_{k_n}, \dots$

a) if $a_{k_n} = O(f(n))$ for $k_n \in \mathbb{N}$

b) $a_{k_n} \leq a_n \leq a_{k_{n+1}}$

all terms with indices $k_n < n < k_{n+1}$

$\Downarrow$
 $a_n = O(f(n))$ for all $n \in \mathbb{N}$.

Let us look at the following case:

(i) $S_n = 1 + \frac{1}{2} + \dots + \frac{1}{n}$

From the theory of series

$$\sum_{k=1}^{\infty} \frac{1}{k} = \lim_{k \rightarrow \infty} \left(1 + \frac{1}{2} + \dots + \frac{1}{k} \right) = +\infty$$

↑
harmonic series

divergent

But how quickly?

ALGORITHMS & COMPLEXITY

LECTURE 4

We choose terms a_n with indices

$$n = 2^k, \quad k \in \mathbb{N}.$$

$$S_{2^0} = S_1 = 1$$

$$S_{2^1} = S_2 = 1 + \frac{1}{2} = S_1 + \frac{1}{2} < S_1 + \frac{1}{2} = 2$$

$$\begin{aligned} S_{2^2} = S_4 &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = S_2 + \frac{1}{3} + \frac{1}{4} < S_2 + \left(\frac{1}{2} + \frac{1}{2}\right) \\ &= S_2 + 1 \\ &< 3 \end{aligned}$$

$$\begin{aligned} S_{2^3} = S_8 &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} \\ &= S_4 + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} < S_4 + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} \\ &= S_4 + 1 < 3 + 1 = 4. \end{aligned}$$

...
looking at the above tendency
we set-up the theoretical hypothesis:

$$\boxed{P(k): S_{2^k} < k+1} \quad \begin{array}{l} \text{obviously} \\ S_n \geq 0. \end{array}$$

$$\text{I. } P(0): S_{2^0=1} < 2$$

$1 < 2. \quad \text{OK}$

$$\text{II. } P(k): S_{2^k} < k+1 \Rightarrow P(k+1): S_{2^{k+1}} < k+2$$

ALGORITHMS & COMPLEXITY

LECTURE 4

$$S_{2^{k+1}} = S_{2^k} + \left(\frac{1}{2^{k+1}} + \frac{1}{2^{k+2}} + \dots + \frac{1}{2^{k+2^k}} \right)$$

$2 \cdot 2^k = 2^{k+1}$

by P(k)

$$< (k+1) + \left(\frac{1}{2^{k+1}} + \frac{1}{2^{k+2}} + \dots + \frac{1}{2^{k+1}} \right)$$

$$\text{but } \left. \begin{array}{l} \frac{1}{2^{k+1}} < \frac{1}{2^k} \\ \frac{1}{2^{k+2}} < \frac{1}{2^k} \\ \vdots \\ \frac{1}{2^{k+1}} < \frac{1}{2^k} \end{array} \right\} \begin{array}{l} 2^k \text{ terms} \\ (*) \end{array}$$

by (*)

$$< (k+1) + \underbrace{\frac{1}{2^k} + \dots + \frac{1}{2^k}}_{\text{times } 2^k} = (k+1) + \frac{2^k}{2^k} = \underline{\underline{k+2}}$$

therefore: a) $\underbrace{S_{2^{k+1}}}_{P(k+1)} < (k+1) + 1.$

Thus by P.M.I. $P(k)$ holds for $\forall k \in \mathbb{N}$

But note that between a_k & a_{k+1} we have terms

$\frac{k}{2^n} \qquad \frac{k+1}{2^{n+1}}$

b) $a_{\frac{k}{2^n}} < a_{\frac{k+1}{2^{n+1}}} < a_{\frac{k+2}{2^{n+1}}} < \dots < a_{\frac{k+1}{2^{n+1}}} < a_{\frac{k+1}{2^{n+1}}}$

ALGORITHMS & COMPLEXITY

LECTURE 4

Thus: since $n = 2^k \Rightarrow k = \lg_2 n$

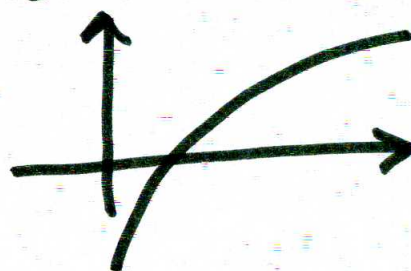
$$a_n = O(k+1)$$

$\Downarrow$ hence

$\forall n \in \mathbb{N}$
since a) & b)

$$\boxed{a_n = O(\lg_2 n)}$$

So the explosion to ∞ is controlled from above by curve (modulo multiplied by constant).



This is slow explosion rate (e.g. slower than linear one).

(ii) For $t_n = n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n}$

$$t_n = \underset{\substack{\uparrow \\ O(n)}}{n} \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right) \underset{\substack{\uparrow \\ O(\lg_2 n)}}{}$$

$$\text{thus } \boxed{t_n = O(n \lg_2 n)}$$

Note that we previously showed that:

ALGORITHMS & COMPLEXITY

LECTURE 4

Thus: since $n = 2^k \Rightarrow k = \lg_2 n$

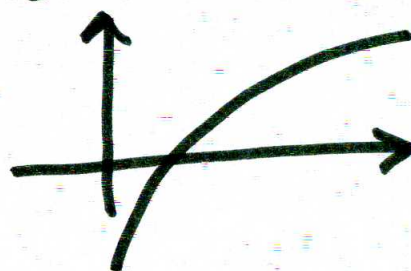
$$a_n = O(k+1)$$

$\Downarrow$ hence

$\forall n \in \mathbb{N}$
since a) & b)

$$\boxed{a_n = O(\lg_2 n)}$$

So the explosion to ∞ is controlled from above by curve (modulo multiplied by constant).



This is slow explosion rate (e.g. slower than linear one).

(ii) For $t_n = n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n}$

$$t_n = \underset{\substack{\uparrow \\ O(n)}}{n} \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right) \underset{\substack{\uparrow \\ O(\lg_2 n)}}{}$$

$$\text{thus } \boxed{t_n = O(n \lg_2 n)}$$

Note that we previously showed that:

ALGORITHMS & COMPLEXITY

LECTURE 4

$$1) \begin{cases} T(1) = A \\ T(n) = 2 T(\lfloor \frac{n}{2} \rfloor) \end{cases} \quad A > 0. \quad \text{implicit form}$$

$\Downarrow$ proved by M.I

$$\begin{cases} a) T(n) = A 2^k \\ b) 2^k \leq n \\ c) k \in \mathbb{N} \text{ max. such that } 2^k \leq n \end{cases} \quad \text{explicit form}$$

It is hard to see from implicit form

$$T(n) = O(?)$$

this is Divide/Conquer type algorithm

But from explicit form we have:

$$T(n) = A 2^k \text{ (as } 2^k \leq n)$$

$$0 \leq \underset{\substack{\uparrow \\ \text{obvious}}}{T(n)} \leq A \cdot n \quad \Rightarrow \quad \boxed{T(n) = O(n)}$$

We can also see that analogously:

$$2) \begin{cases} T(1) = A \\ T(n) = 4 T(\lfloor \frac{n}{2} \rfloor) \end{cases} \xRightarrow{\text{I.F.}} \begin{cases} T(n) = A 4^k \\ 2^k \leq n \\ k \text{ max s.t. } 2^k \leq n \end{cases} \quad \text{E.F.}$$

Thus

$$0 \leq T(n) = A \cdot 4^k = A \cdot 2^k \cdot 2^k \leq A \cdot n^2 \quad \Rightarrow \quad \boxed{T(n) = O(n^2)}$$

ALGORITHMS & COMPLEXITY

LECTURE 4

And in general

E.F

$$\text{I.F. } \begin{cases} T(1) = A \\ T(n) = (B=2^L) \cdot T(\lfloor \frac{n}{2} \rfloor) \end{cases} \Rightarrow \begin{cases} a) T(n) = A \cdot (B)^k \\ b) 2^k \leq n \\ c) k \text{ max } 2^k \leq n \end{cases}$$

$$0 \leq T(n) = A \cdot B^k = A \underbrace{(2 \cdot 2 \cdots 2)}_{L\text{-times}}^k$$

$$= A \underbrace{2^k \cdot 2^k \cdots 2^k}_{L\text{-times}} \leq A \underbrace{n \cdot n \cdots n}_{L\text{-times}}$$

$$\Rightarrow T(n) = O(n^L)$$

If $B \neq 2^L$ e.g. $B=3$

$$\text{I.F. } \begin{cases} T(1) = A \\ T(n) = 3 T(\lfloor \frac{n}{2} \rfloor) \end{cases} \Rightarrow \begin{cases} a) T(n) = A \cdot 3^k \\ b) 2^k \leq n \\ c) k \text{ - max. such that } 2^k \leq n \end{cases}$$

then

$$0 \leq T(n) = A \cdot 3^k = A (2^{\log_2 3})^k = A \cdot 2^{k \log_2 3}$$

$$\text{but } 3 = 2^{\log_2 3} \Rightarrow A (2^k)^{\log_2 3}$$

$$\leq A n^{\log_2 3} \quad (b)$$

$$\Downarrow \\ \boxed{T(n) = O(n^{\log_2 3})}$$

Note $1 < \log_2 3 < 2$

so we have
order of explosion faster than linear & slower than quadratic

ALGORITHMS & COMPLEXITY

LECTURE 4

We showed that to examine the complexity of algorithms (and possibly sharpness)

- calculus of integration is needed sometimes (e.g. $a_n = \ln n!$)
- or series theory to assess exactly the sum or its upper bound e.g.:

$$a_0(1+q+q^2+\dots+q^n+\dots) = \frac{a_0}{1-q}$$

(geometric progression) $|q| < 1$

sum

$$t_n = 1 + \frac{1}{2} + \dots + \frac{1}{n} \leq \text{upper bound was found.}$$

In general exact counting of sums can be difficult. Sometimes

a telescoping technique can be useful:

Indeed observe

$$S_m = \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots + \frac{1}{(n-1)n} + \frac{1}{n(n+1)}$$

ALGORITHMS & COMPLEXITY

If $a_k = \frac{1}{k(k+1)} \stackrel{\uparrow}{=} \frac{1}{k} - \frac{1}{k+1}$
observe that

Thus

$$S_n = \left(\frac{1}{1} - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \left(\frac{1}{3} - \frac{1}{4}\right) + \dots + \left(\frac{1}{n-1} - \frac{1}{n}\right) + \left(\frac{1}{n} - \frac{1}{n+1}\right) = 1 - \frac{1}{n+1}$$

$$S_n = 1 - \frac{1}{n+1} \Rightarrow S_n = \Theta(1)$$

One can generalize that technique to the schemes involving sums

• $\sum_{k=0}^{\infty} \frac{1}{a_k a_{k+1}}$ provided $a_{k+1} - a_k = r$ (arithmetic seq.)
 $= \lim_n S_n$
 $\left(\frac{1}{a_0 a_1} + \dots + \frac{1}{a_n a_{n+1}}\right)$

Note

$$\frac{1}{a_k} - \frac{1}{a_{k+1}} = \frac{a_{k+1} - a_k}{a_k a_{k+1}} = \frac{r}{a_k a_{k+1}}$$

$$\frac{1}{a_k a_{k+1}} \stackrel{\Downarrow}{=} \frac{1}{r} \left(\frac{1}{a_k} - \frac{1}{a_{k+1}} \right)$$

ALGORITHMS & COMPLEXITY

LECTURE 4

$$S_m = \frac{1}{a_0 a_1} + \frac{1}{a_1 a_2} + \frac{1}{a_2 a_3} + \dots + \frac{1}{a_{n-1} a_n} + \frac{1}{a_n a_{n+1}}$$

$$= \frac{1}{r} \left(\frac{1}{a_0} - \frac{1}{a_1} \right) + \frac{1}{r} \left(\frac{1}{a_1} - \frac{1}{a_2} \right) + \frac{1}{r} \left(\frac{1}{a_2} - \frac{1}{a_3} \right) + \dots + \frac{1}{r} \left(\frac{1}{a_{n-1}} - \frac{1}{a_n} \right) + \frac{1}{r} \left(\frac{1}{a_n} - \frac{1}{a_{n+1}} \right)$$

$$= \frac{1}{r} \left(\frac{1}{a_0} - \frac{1}{a_{n+1}} \right)$$

In the 1st example $\frac{1}{n(n+1)} \leftarrow$ special case

$$a_k = k \quad a_{k+1} = k+1 \quad a_{k+1} - a_k = 1 = r$$

But telescoping can also be applied to other sums different than (A):

$$\sum_{n=1}^{\infty} \left(\sqrt[n]{n} - \sqrt[n+1]{n+1} \right) = \lim_n S_m$$

$$S_m = \left(\sqrt[1]{1} - \sqrt[2]{2} \right) + \left(\sqrt[2]{2} - \sqrt[3]{3} \right) + \dots + \left(\sqrt[n]{n} - \sqrt[n+1]{n+1} \right) \\ = 1 - \sqrt[n+1]{n+1}$$

One can apply the above telescoping method to the following examples:

ALGORITHMS & COMPLEXITY

LECTURE 4

$$a) \sum_{k=1}^n \frac{1}{(2k-1)(2k+1)} = \frac{n}{2n+1}$$

$$b) \sum_{k=1}^n \frac{1}{(3k-2)(3k+1)} = \frac{n}{3n+1}$$

$$c) \sum_{k=0}^{n-1} \frac{1}{(n+k)(n+k+1)} = \frac{1}{2n}$$

$$d) \sum_{k=0}^{n-1} \frac{1}{(n-2k)(n-2k+2)} = \frac{n}{4-n^2}$$

↑ for n odd.

Recall that in the analysis of algorithms we wish to (for a_n)

- compute exactly the number a_n (at best) which counts e.g. memory consumption, number of iterations, time execution etc. This however (the exact count) is usually difficult.

ALGORITHMS & COMPLEXITY

LESSURE 4

- then if a_n cannot be computed exactly we try first to find an upper bound in terms of:

$$a_n = O(f(n))$$

- once the latter is found with naive method we try to improve it with more advance method (which not always gives such improvement) so that:

$a_n = O(g(n))$ where $g(n) = O(f(n))$
Any improvement is the step in a good direction.

- in the process of such improvement we keep in mind the ultimate goal:

$$a_n = \Theta(h(n))$$

"sharp estimate"

Here computers can help as if

we have $a_n = O(f(n))$ we can plot

$(n, kf(n))$ against (n, a_n) and try

↑ constant

to see whether we can improve or not our estimate.

ALGORITHMS & COMPLEXITY

LECTURE 4

Example:

a) For Bubble sort

$$a_n = 1 + 2 + \dots + n - 1$$

(i) if naive counting is used

$$0 \leq a_n = 1 + 2 + \dots + n - 1 \leq \underbrace{n + n + \dots + n}_{n-2} + n$$

$$0 \leq a_n \leq n(n-1) (*) \Rightarrow \boxed{O(n^2) = a_n} (**)$$

(ii) But the exact counting yields

$$(**) a_n = \frac{n(n-1)}{2} \quad \text{so it is better (modulo constant) than } (*)$$

But it also shows $\boxed{a_n = \Theta(n^2)}$

In this example since exact result was "computable" we have indication from

$(**)$ about sharp estimate immediately. (naive method gives that)

b) take now $S_n = 1 + \frac{1}{2} + \dots + \frac{1}{n}$
here it is hard to count S_n

(i) naive estimation

$$0 \leq S_n \leq 1 + 1 + \dots + 1 = n$$

$$\Rightarrow \boxed{S_n = O(n)}$$

ALGORITHMS & COMPLEXITY

LECTURE 4

- (ii) more advanced estimation
(if you plot (n, S_n) against (n, n) one can see that S_n is well-below n).

In this lecture we showed (with trickier proof)

$$S_n = O(\lg_2 n)$$

which is better as $\lg_2 n = O(n)$

So naive method did not give us a sharp result.

Is now $S_n = O(\lg_2 n)$ sharp?

$$\text{i.e. } S_n = \Theta(\lg_2 n)$$

A possible tool is to use the "integration approach" which can prove both

$$(i) \quad 0 \leq S_n \leq M_2 \lg_2 n \Rightarrow S_n = O(\lg_2 n)$$

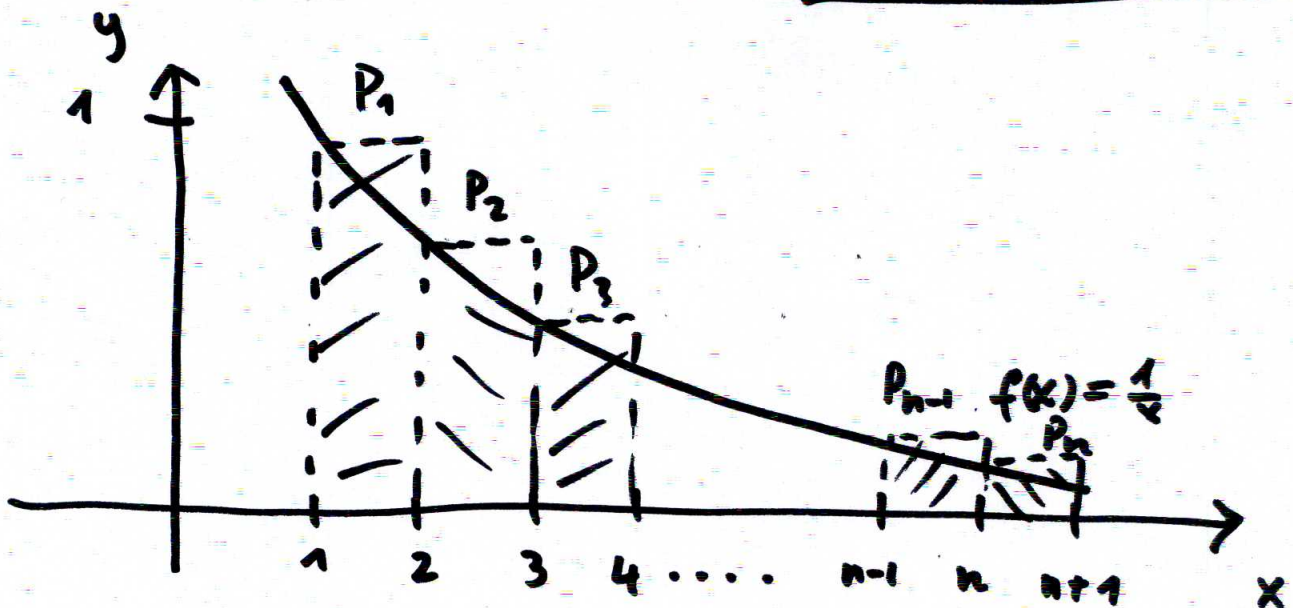
$$(ii) \quad M_1 \lg_2 n \leq S_n \Rightarrow (i) \text{ \& } (ii) \Rightarrow S_n = \Theta(\lg_2 n)$$

We only show (ii) as (i) was proved by different technique & follows also analogously as in (ii).

ALGORITHMS & COMPLEXITY

LECTURE 4

We show now sharpness $[S_n = \Theta(\lg_2 n)]$



$P_i \leftarrow$ rectangle A_i - area of P_i

From picture it is clear that

$A_1 + A_2 + \dots + A_n \geq$ area of $\int_1^{n+1} \frac{1}{x} dx$

$$(2-1) \frac{1}{1} + (3-2) \frac{1}{2} + (4-3) \frac{1}{3} + \dots + [n-(n-1)] \frac{1}{n-1} + (n+1-n) \frac{1}{n} \geq \int_1^{n+1} \frac{1}{x} dx$$

$$\underbrace{1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n-1} + \frac{1}{n}}_{S_n} \geq \ln x \Big|_1^{n+1} = \ln(n+1) - \ln 1 = \ln(n+1)$$

$$\underline{\underline{S_n \geq \ln(n+1) \geq \ln(n) = \lg_2 n}}$$

but $\ln n = \lg_e n = \lg_e 2 \cdot \lg_2 n \Rightarrow \underline{\underline{S_n = \Theta(\lg_2 n)}}$